

Hybrid Motion-Gated Person Detection: Guarantees, Failure Modes and a Stabilised Gate

Yana Osinchuk

Hochschule Darmstadt – University of Applied Sciences

September 2026

Abstract

Running a convolutional person detector on every frame of a static-camera stream wastes most of its computation whenever the scene does not change. We revisit a hybrid system that gates YOLOv8m with frame differencing and turn it into a design with guarantees and understood failure modes. A scheduler with a refresh interval K and a minimum interval M bounds the age of every reported box by $K - 1$ frames and keeps the fraction of frames sent to the detector between $1/K$ and $1/M$. We show that the classic difference gate suffers from a rectification bias of 1.128σ under sensor noise; a percolation argument predicts its breakdown at $\sigma = 13.9$, and we measure 13.9. A stabilised gate that smooths signed differences, registers frames by phase correlation under an edge-based model selection, compensates illumination offsets and propagates the registration error into its threshold raises the noise limit to $\sigma = 33.8$ and cuts false triggers from 0.206 to 0.014 of the transitions on a controlled benchmark with exact ground truth. Gated by it, the detector runs on 0.542 of the frames while the system reproduces the every-frame output (recall 0.997, mean IoU 0.981). At smaller budgets the rate-limited gate keeps a recall of 0.971 where fixed-rate subsampling reaches 0.883. On a busy real video the gate is active on 0.994 of the frames, and M bounds the cost. Code, tests and a one-command reproduction accompany the paper.

Keywords: video analytics; person detection; YOLOv8; frame differencing; motion gating; background subtraction; image registration; efficient inference

1 Introduction

Person detection is the entry point of many video-analytics applications: occupancy counting, safety monitoring, football analysis and smart-building control. One-stage detectors of the YOLO family [8, 16] deliver accurate boxes in a single forward pass, but that pass is expensive on the CPUs and embedded boards on which camera streams are often processed. On one CPU thread of our test machine, YOLOv8m needs 698 ms per 640×640 frame, more than ten times the frame period of a 30 FPS stream. At the same time, a static camera observes the same background for most of the day, so running the detector on every frame mostly recomputes an answer that has not changed.

The obvious remedy is a *motion gate*: a cheap change detector inspects every frame, the expensive detector runs only when change is reported, and the last detections are reused otherwise. Frame differencing [6] or background subtraction [17, 21] costs about a millisecond per frame, and difference-based frame filtering is a core ingredient of video-analytics systems [1, 9, 11]. Gating is not free, however. It introduces a new kind of error, *staleness*: reused boxes lag behind the scene. And the gate can fail in two directions. False triggers caused by sensor noise, camera jitter or illumination changes waste detector calls and erode the savings; missed triggers caused by small or slow objects, or by a person leaving the image, leave outdated boxes.

This paper revisits a prototype of such a hybrid system, which gated YOLOv8m with OpenCV frame differencing and reported a $1.71\times$ speed-up on a small synthetic benchmark. In that benchmark the gated and the ungated system were both perfect, the sensitivity ablation of the

gate was flat, and the one scenario in which the gate failed (camera jitter) remained unexplained, so the evaluation could not say *when* gating is safe. We turn the prototype into an analysed and tested design and make four contributions:

- **A scheduler with guarantees.** Two interpretable parameters, a refresh interval K and a minimum interval M , bound the age of every reported detection by $K - 1$ frames and the fraction of frames sent to the detector between $1/K$ and $1/M$, whatever the scene (Section 3.2). A cost model yields the speed-up and the break-even condition and is validated end to end.
- **An analytical failure envelope of frame differencing.** The original gate takes absolute differences *before* smoothing. Noise therefore contributes a mean of 1.128σ that smoothing cannot remove, a *rectification bias*; combined with a percolation argument, it predicts the noise level at which the gate breaks down. Simple models predict the response to illumination steps and to small or slow objects (Section 3.4). All predictions are tested experimentally.
- **A stabilised gate (FD-S).** Smoothing a *signed* difference, registering consecutive frames by phase correlation guarded by an edge-based model selection, compensating global intensity offsets and propagating the registration error into the threshold removes most false triggers caused by noise, illumination changes and camera jitter, at a cost of a few milliseconds per frame (Section 3.5).
- **An evaluation protocol and an open implementation.** A controlled benchmark with exact ground truth isolates nuisance factors one at a time, every policy is compared with fixed-rate subsampling at the same computational budget, and a real video serves as a held-out check. The Python package, a command-line tool, unit tests and a single script that regenerates every number, table and figure of this paper accompany it.

From a practical point of view, the analysis leads to simple deployment rules (Section 7): choose K from the staleness the application tolerates, use M to cap the cost in busy scenes, and use the stabilised gate whenever the camera may vibrate or the lighting may change.

2 Related work

Object detection. One-stage detectors such as YOLO [16] and its Ultralytics successor YOLOv8 [8], trained on COCO [12], predict boxes and classes in a single network pass. They process frames independently and do not exploit temporal redundancy. We use YOLOv8m as a black box; nothing in the method depends on the particular detector.

Exploiting temporal redundancy. One line of work reuses computation inside the network: Deep Feature Flow [20] computes expensive features on sparse key frames and propagates them with optical flow. Another line filters frames in front of the network. NoScope [9] cascades difference detectors and specialised models for binary video queries, Chameleon [7] adapts frame rate and resolution over time, Reducto [11] filters frames on the camera using low-level difference features with query-specific thresholds, and FrameHopper [1] learns a frame-skipping policy by reinforcement learning. These systems target multi-stream or query-driven analytics and optimise end-to-end query accuracy. We isolate the single-stream case, make the trade-off between cost and staleness explicit with provable bounds, and characterise the physical failure modes of the change detector itself. Tracking-by-detection, for example SORT [2], is complementary: a tracker could extrapolate boxes between detector calls instead of retaining them.

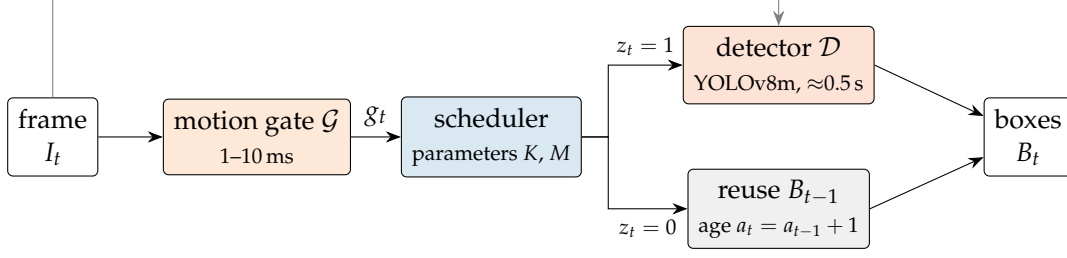


Figure 1: System overview. A cheap gate inspects every frame. The scheduler calls the detector on the first frame, when motion is pending and at least M frames have passed since the last call, or when K frames have passed without a call; otherwise the previous boxes are reported again and their age grows by one.

Change detection. Frame differencing is the oldest change detector [6]. Adaptive Gaussian mixtures [17], available in OpenCV [3] as MOG2 [21], maintain a per-pixel background model. The survey of Garcia-Garcia et al. [5] lists illumination changes, camera jitter and noise among the main open challenges of background subtraction, which are precisely the factors analysed here. We use MOG2 as a stronger but costlier baseline gate.

Image registration. Global translations are classically estimated by phase correlation [10], which is non-iterative and insensitive to global intensity changes. Iterative methods such as Lucas-Kanade [13] or enhanced-correlation-coefficient maximisation [4] are more accurate for sub-pixel motion but more expensive. The stabilised gate uses phase correlation and guards it with a model-selection test.

3 Method

3.1 Problem setting

A static camera delivers frames $I_0, \dots, I_{T-1}$. A detector $\mathcal{D}$ maps a frame to a set of person boxes at cost C_y per call. A gate $\mathcal{G}$ maps the pair (I_{t-1}, I_t) to a decision $g_t \in \{0, 1\}$ at cost C_m per frame ($g_0 = 0$). A scheduler decides $z_t \in \{0, 1\}$, and the system reports $B_t = \mathcal{D}(I_t)$ if $z_t = 1$ and $B_t = B_{t-1}$ otherwise (Figure 1). We call $r = \frac{1}{T} \sum_t z_t$ the *invocation ratio* and $a_t = t - \ell_t$ the *age* of the reported boxes, where $\ell_t = \max\{s \leq t : z_s = 1\}$ is the most recent detector call.

3.2 Scheduler

Let p_t indicate that motion has been reported since the last call, $p_t = g_t \vee (p_{t-1} \wedge \neg z_{t-1})$ with $p_{-1} = 0$. The scheduler calls the detector if

$$z_t = \mathbf{1}[t = 0 \vee (p_t \wedge t - \ell_{t-1} \geq M) \vee t - \ell_{t-1} \geq K], \quad 1 \leq M \leq K. \quad (1)$$

The refresh interval K bounds staleness; the minimum interval M caps the call rate. A trigger that arrives too early is delayed, never dropped. For $M = 1$ the rule reduces to the original design $z_t = \mathbf{1}[t = 0 \vee g_t \vee t - \ell_{t-1} \geq K]$.

Proposition 1. *For every gate sequence (g_t) and $1 \leq M \leq K$: (i) $a_t \leq K - 1$ for all t ; (ii) if $g_s = 1$, the detector is called at some $t \in \{s, \dots, s + M - 1\}$; (iii) consecutive calls are at least M and at most K frames apart, hence $\lceil T/K \rceil \leq \sum_t z_t \leq \lceil T/M \rceil$, and both bounds are attained (gate never and always active).*

Proof. (i) The refresh clause fires as soon as $t - \ell_{t-1} = K$, so gaps between calls never exceed K frames and ages within a gap are at most $K - 1$. (iii) Apart from $t = 0$, every clause of (1) requires $t - \ell_{t-1} \geq M$ because $M \leq K$; the upper gap bound is (i). (ii) Let $\ell < s$ be the last call before s . The pending flag is set at s and stays set until the next call, which happens no later than the first $t \geq s$ with $t - \ell \geq M$, i.e. at $\max(s, \ell + M) \leq s + M - 1$. $\square$

In practice K follows from a staleness budget $s_{\max}$ at frame rate f as $K = \lfloor s_{\max} f \rfloor + 1$. Under continuous motion the scheduler runs the detector on every M -th frame, exactly like fixed-rate subsampling, whereas static periods cost only one call per K frames.

Cost model. With per-frame costs C_m and C_y , the mean cost per frame is $C_m + rC_y$, so the speed-up over running the detector on every frame and the break-even condition are

$$S = \frac{C_y}{C_m + rC_y}, \quad S > 1 \iff r < 1 - \frac{C_m}{C_y}. \quad (2)$$

For our gates C_m/C_y lies between 0.0020 (FD) and 0.0151 (MOG2), so the gate's own cost is negligible and the savings are governed by r , i.e. by the activity of the scene and the gate's false-trigger rate. Every false trigger costs a full detector call; every missed trigger costs accuracy, which Proposition 1(i) limits to $K - 1$ frames of staleness.

3.3 The frame-differencing gate (FD)

The original gate follows the classic OpenCV recipe [3]:

$$D_t = G * \sum_c w_c |I_{t,c} - I_{t-1,c}|, \quad \text{mask}_t = \mathbf{1}[D_t > \tau], \quad (3)$$

with the BT.601 grey weights $w = (0.114, 0.587, 0.299)$ in BGR order and a 5×5 Gaussian kernel G . The mask is dilated d times with a 3×3 element, and the gate is active if an external contour encloses at least $A_{\min}$ pixels. We keep the original parameters $\tau = 20$, $A_{\min} = 900$ px and $d = 3$ throughout.

3.4 Failure envelope of the frame-differencing gate

Sensor noise. Assume independent $\mathcal{N}(0, \sigma^2)$ noise per channel, pixel and frame, and a static scene. A channel difference is $\mathcal{N}(0, 2\sigma^2)$, and its absolute value has mean $2\sigma/\sqrt{\pi}$ and variance $2\sigma^2(1 - 2/\pi)$. The grey conversion keeps the mean ($\sum_c w_c = 1$) and scales the variance by $\sum_c w_c^2 = 0.4470$; smoothing keeps the mean and scales the variance by $\sum G^2 = 0.0748$. Hence, approximately,

$$D_t \sim \mathcal{N}(1.128\sigma, (0.156\sigma)^2). \quad (4)$$

Smoothing suppresses the fluctuations but not the mean, which the absolute value has created: a *rectification bias*. The per-pixel exceedance probability is $p_{FD}(\sigma) = 1 - \Phi((\tau - 1.128\sigma)/(0.156\sigma))$.

When does the gate fire? The dilations turn every exceeding pixel into a $(2d + 1) \times (2d + 1)$ square. Randomly placed aligned squares of side L form large connected clusters once their density reaches η_c/L^2 with $\eta_c \approx 1.0988$ [14]. For $d = 3$ this is $p_c = 0.0224$: below p_c clusters rarely reach $A_{\min}$, above it they grow without bound and the gate fires in every frame. The onset is therefore predicted where $p(\sigma^*) = p_c$, which gives $\sigma_{FD}^* = 13.9$ (13.4–15.1 for $p \in [0.01, 0.1]$). Smoothing correlates neighbouring exceedances, so this is an estimate that the experiments must confirm.

Illumination. A uniform intensity step Δ changes every unsaturated pixel by $|\Delta|$, so FD fires if and only if $|\Delta| > \tau$, whatever the scene content.

Object size and speed. A textured silhouette of height h that moves by v pixels changes at least a leading and a trailing strip of about $v \times h$ pixels. After dilation a strip covers $(v + 2d)(h + 2d)$ pixels, so the gate fires if

$$v \geq v^* = \frac{A_{\min}}{h + 2d} - 2d. \quad (5)$$

For the person heights of our benchmark, $h = 68, 113$ and 317 px, the bound is $v^* = 6.2, 1.6$ and -3.2 px/frame; a negative value means that every displacement triggers the gate. Interior texture and merging strips only add changed pixels, so the model is conservative.

Camera jitter. A translation δ changes a pixel by about $|\nabla I \cdot \delta|$. Requiring that $A_{\min}$ pixels exceed τ gives the order-of-magnitude estimate $\delta^* \approx \tau/g_q$, where g_q is the gradient magnitude exceeded by $A_{\min}$ pixels of the smoothed background. For our background $g_q = 53.6$ grey levels per pixel and $\delta^* \approx 0.37$ px: scenes with sharp edges trigger FD for sub-pixel camera motion.

3.5 The stabilised gate (FD-S)

Each step of FD-S addresses one failure mode of FD:

1. *Signed smoothing (noise).* $G_t = G * \text{gray}(I_t)$ is computed in floating point and the residual $R_t = G_t - \hat{G}_{t-1}$ keeps its sign, so noise has zero mean and standard deviation $\sigma\sqrt{2\sum_c w_c^2 \sum G^2} = 0.259\sigma$. With $p_{FD-S}(\sigma) = 2(1 - \Phi(\tau/(0.259\sigma)))$ the percolation criterion predicts $\sigma_{FD-S}^* = 33.9$ (band 30.0–47.0), more than twice the tolerance of FD.
2. *Registration (jitter).* The translation s_t between G_{t-1} and G_t is estimated by phase correlation [10] on half-resolution images with a Hann window. Estimates with a weak peak (response < 0.05), an implausible magnitude (> 20 px) or a magnitude below the accuracy of the estimator (< 0.1 px) are discarded. $\hat{G}_{t-1}$ is G_{t-1} warped by s_t with cubic interpolation; bilinear warping would blur only the reference frame and leave residuals along every edge.
3. *Model selection.* Phase correlation assumes that the background dominates the image, and a large, high-contrast moving object can capture the correlation peak. The shift is therefore accepted only if it lowers the median absolute residual on the 10% strongest-gradient pixels by at least 10% compared with $s = 0$. Registration can only be verified on image structure (the aperture problem), and on these pixels the background dominates, so a moving object cannot pull the decision.
4. *Illumination.* The median β_t of R_t is subtracted. While $|\beta_t| > 2$, pixels within 6 grey levels of 0 or 255 in either frame are ignored, because clipped values cannot follow a global offset. Otherwise they are kept, so that a saturated (e.g. white) moving object remains visible.
5. *Error propagation.* A registration error e leaves a residual of about $e \cdot \nabla G$. After a shift has been compensated, a pixel counts as changed only if $|R_t - \beta_t| > \tau + \varepsilon|\nabla G_t|$ with $\varepsilon = 0.5$ px, the order of the sub-pixel accuracy of half-resolution phase correlation. Without compensation the plain threshold τ applies, so a static camera keeps full sensitivity. A border of $\lceil |s_t| \rceil + 1$ pixels is ignored.
6. Dilation, contours and $A_{\min}$ as in FD.

FD-S costs 7.25 ms per 640×640 frame against 1.43 ms for FD. It models global translation only; rotation, zoom, parallax and a foreground that covers most of the image violate its assumptions.

3.6 Baseline gate: MOG2

As a stronger baseline we use OpenCV’s adaptive Gaussian mixture model [21] (history 500, variance threshold 16, shadows ignored), followed by a 3×3 morphological opening and the same contour rule. It adapts to gradual changes but needs several frames to absorb sudden ones, and it costs 10.51 ms per frame.

4 Implementation

The method is implemented as the Python package `motiongate` (Python ≥ 3.10 , NumPy and OpenCV; Ultralytics only for the detector wrapper). Its modules mirror Section 3: `gates` provides FD, FD-S and MOG2 behind one interface, `scheduler` implements rule (1), `detector` wraps Ultralytics (any callable that returns boxes can replace it), `theory` contains the closed-form predictions of Section 3.4, and `cli` is the command-line tool. A minimal example:

```
from motiongate import (MotionGatedDetector, SchedulerConfig,
                        UltralyticsPersonDetector, make_gate)

system = MotionGatedDetector(
    UltralyticsPersonDetector("yolov8m.pt"),
    make_gate("stabilized"),
    SchedulerConfig.from_max_staleness(seconds=0.5, fps=30))
for frame in frames:                                # BGR uint8 images
    result = system.process(frame)
    print(result.index, result.reason, result.detection_age, result.detections)
```

The command-line tool processes a video file or a camera and writes an annotated video and a per-frame CSV log, e.g. `motiongate input.mp4 -max-staleness 0.5 -output out.mp4 -log frames.csv`.

Several design decisions matter in deployment. Input frames are never modified. A change of resolution restarts the stream, because the gate compares aligned frames. Every result states whether its boxes are fresh or retained, why the detector did or did not run (initial frame, motion, refresh, skipped) and how old the boxes are, so that downstream logic can, for instance, treat stale boxes differently. The package comes with 28 unit tests covering the behaviour of the gates on synthetic textures (static scenes, moving objects, translations, illumination steps, noise), the guarantees of Proposition 1, decision-by-decision equivalence of the online scheduler and the offline replica used in the experiments, and agreement of the noise model (4) with the actual OpenCV pipeline.

5 Experimental setup

5.1 Controlled benchmark

Real surveillance footage rarely comes with the information needed to test a gate: when exactly something moved, by how much, and whether the camera moved. We therefore build a controlled benchmark from one real photograph, following the construction of the original prototype. All persons in Ultralytics’ `bus.jpg` are segmented with YOLOv8n-seg, the dilated union of their masks is filled by Telea inpainting [18], and a 640×640 crop serves as background. The fully visible, most upright pedestrian is cut out with its mask and composited back at scripted positions, either at full scale (height 317 px) or as a distant walker (113 px). An alpha-blended cut-out on an inpainted background is not photo-realistic, but presence, motion and the target box are known exactly. Every frame receives Gaussian sensor noise with $\sigma = 1.5$ per channel. Camera

Table 1: Scenarios of the controlled benchmark (90 frames each).

Scenario	Content (and what it tests)
Walk	person walks across the scene at 6 px/frame (sustained motion)
Slow walk	1.5 px/frame (weak motion signal)
Stop and go	walks for 30 frames, stands for 30, walks again (idle period)
Enter, stop, exit	empty for 10 frames, enters from the left, stands for 40 frames, leaves to the right, empty for the last 5 frames (entry and exit latency)
Stationary person	person present but motionless (the gate must stay silent)
Distant walker	person of height 113 px walking at 2 px/frame (area filter)
Empty scene	background and noise only
Illumination steps	empty scene; global steps of +32 and -50 grey levels at frames 30 and 60
Camera jitter	empty scene; independent translations of up to ± 3 px per axis and frame
Jitter + walk	walking person filmed by a jittering camera

jitter is rendered by extracting the frame at a sub-pixel offset from a larger canvas with Lanczos interpolation, so that, as with a physical camera, the border shows real scene content and the translation does not blur the image.

Table 1 lists the 10 scenarios of 90 frames each (3 s at 30 FPS, 900 frames in total). The ground truth is the tight box of the visible part of the cut-out; a transition counts as motion if a visible person changed position.

5.2 Metrics

Gate level. For every transition $(t-1, t)$ the gate decision is compared with the motion label: $\text{recall} = P(g_t = 1 \mid \text{motion})$, false-positive rate $(\text{FPR}) = P(g_t = 1 \mid \text{no motion})$ and precision. Statistics are pooled over 10 independent noise and jitter realisations (8900 transitions per gate) and reported with Wilson 95% intervals [19].

System level. A frame with a visible person counts as a hit if a reported box overlaps the ground truth with $\text{IoU} \geq 0.5$ (one-to-one matching); frame recall is the fraction of hits among the 615 frames with a visible person. On the 285 frames without a person, a frame on which any box is reported is a *stale frame*. We also report the mean IoU of hits, the maximum age, and entry and exit latencies in the enter-stop-exit scenario, i.e. the number of frames until a policy reports the entering person, or stops reporting a box after the person has left, relative to running the detector on every frame.

Cost. We report the invocation ratio r , the estimated run time (the sum of the measured per-frame gate costs and the measured detector costs of the invoked frames) and the speed-up relative to the detector on every frame.

5.3 Policies and protocol

We compare (i) the detector on every frame; (ii) fixed-rate subsampling, i.e. every N -th frame with $N \in \{1, \dots, 45\}$, which spends a fixed budget without looking at the content; and (iii) gated policies with FD, FD-S or MOG2, $K \in \{5, 10, 15, 30, 45, 90\}$ and $M = 1$ unless stated otherwise. The default is $K = 15$, i.e. 0.5 s at 30 FPS. Because a policy only selects which detector outputs are reported, YOLOv8m (confidence 0.25, input size 640) is run once on every frame of the first realisation and every policy is replayed offline with the exact rule (1). A unit test verifies that the replay reproduces the online scheduler decision by decision, and an end-to-end run of the online system with FD and FD-S validates both the decisions and the cost model.

Table 2: Gate-level results on the controlled benchmark, pooled over 10 realisations (8900 transitions per gate). Recall and FPR are fractions of the transitions with and without motion; the per-scenario columns single out the hardest cases. Costs per 640×640 frame on one CPU thread.

Gate	Recall (all)	FPR (all)	Precision (all)	Recall distant walker	FPR illumination	FPR jitter	Cost (ms/frame)
FD (original)	0.996	0.206	0.832	1.000	0.022	0.996	1.43
FD-S (stabilised)	0.996	0.014	0.987	1.000	0.000	0.069	7.25
MOG2	0.944	0.114	0.895	0.955	0.236	0.226	10.51

5.4 Operating-envelope sweeps

To test the predictions of Section 3.4, one nuisance factor is varied at a time with the gates in isolation: noise with $\sigma \in [0, 48]$, illumination steps $\Delta \in [0, 80]$ and inter-frame camera translations in $[0, 8]$ px on the empty scene, and a person of height 68, 113 or 317 px walking at 1–10 px/frame. Every point uses 3 realisations of 30 transitions.

5.5 Real video

As a held-out check we use OpenCV’s sample `vtest.avi` [15]: 795 frames of 768×576 pixels at 10 FPS from a static camera overlooking a pedestrian area, with 6.0 detected persons per frame on average. The video has no annotations. The reference is therefore YOLOv8m run on every frame, and we measure how well a policy reproduces it: recall, precision and F1 of reported boxes against reference boxes ($\text{IoU} \geq 0.5$). This quantifies agreement with the detector, not detection accuracy. We use $K = 10$ (1 s), $M \in \{1, \dots, 6\}$ and fixed rates $N \in \{1, \dots, 10\}$. The design of FD-S was frozen on the controlled benchmark before it was run on this video.

5.6 Environment

All timings were measured on a single CPU thread (PyTorch 2.6, Ultralytics 8.3.235, OpenCV 4.13). Absolute times depend on the hardware, whereas ratios such as r and C_m/C_y transfer better. One script regenerates every result, table and figure (Appendix A).

6 Results

6.1 Gate level

Table 2 summarises the gates over 8900 transitions each. The original gate detects almost every motion transition (recall 0.996) but triggers on 0.206 of the transitions without motion, almost entirely because of camera jitter, where it fires on 0.996 of them. FD-S keeps the recall (0.996), reduces the false-positive rate to 0.014 (95% interval $[0.011, 0.018]$) and raises the precision from 0.832 to 0.987. MOG2 misses more motion (recall 0.944) because it needs frames to build its model and partly absorbs slow motion; it adapts to illumination steps only gradually, and it is the most expensive gate (10.51 ms per frame).

The sweeps in Figure 2 test the predictions of Section 3.4:

- *Noise.* FD switches from never to always active at $\sigma = 13.9$ (50% crossing), where the percolation criterion predicts 13.9. FD-S switches at $\sigma = 33.8$, predicted 33.9. The rectification bias thus accounts quantitatively for the difference in noise tolerance.

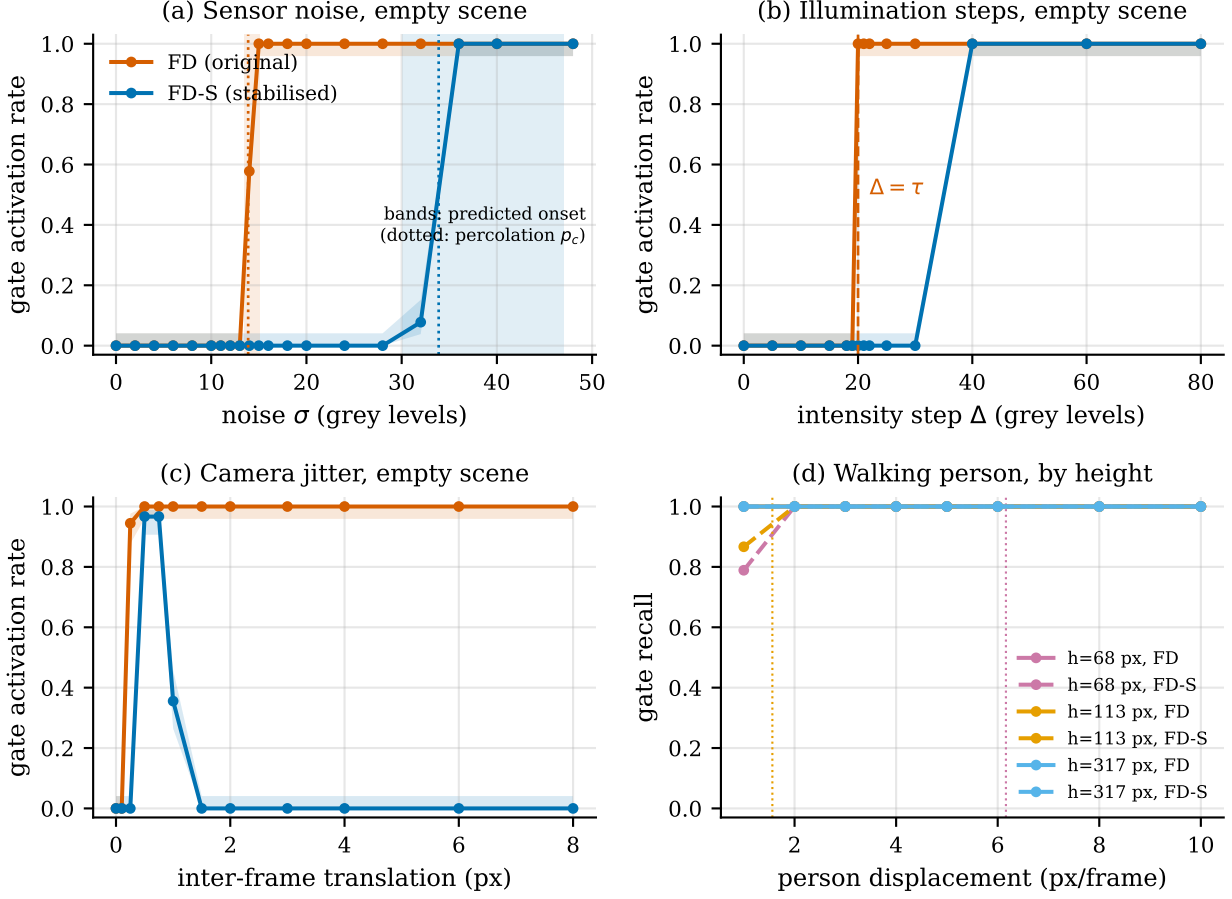


Figure 2: Operating envelope of FD and FD-S (3 realisations $\times$ 30 transitions per point; shaded: 95% Wilson intervals). (a) Noise; bands mark the predicted onset for per-pixel exceedance probabilities between 0.01 and 0.1, dotted lines the percolation prediction. (b) Illumination steps; the dashed line marks $\Delta = \tau$. (c) Camera translation between consecutive frames. (d) Walking persons of three heights; dotted lines mark the area-model bound v^* where it is positive.

- *Illumination.* FD fires from $\Delta = 19.5$ on, which matches the prediction $\Delta > \tau = 20$ up to the integer rounding of the OpenCV pipeline. FD-S tolerates steps of up to 30 grey levels and fires beyond (crossing at 35.0). For large steps, the neighbourhoods of saturated regions change by less than the offset, and after smoothing such partially clipped pixels escape the saturation mask. The illumination scenario of the benchmark (steps of +32 and -50) does not trigger FD-S, but its immunity is not complete.
- *Camera jitter.* FD fires from translations of 0.18 px on, the same order of magnitude as the estimate $\delta^* \approx 0.37$ px. FD-S suppresses translations of 1.5 px and more completely but fires in a band around 0.5–1 px. Such shifts already exceed the threshold at strong edges, yet they are too small for a reliable half-resolution estimate, so the compensation is rejected or inaccurate. This band explains the residual false-positive rate of 0.069 in the jitter scenario.
- *Size and speed.* All three persons trigger FD already at 1 px/frame, far below the area-model bounds of 6.2 and 1.6 px/frame for heights of 68 and 113 px. The textured interior of the silhouette changes many more pixels than the two edge strips, so the bound is tight only for weakly textured objects. FD-S is slightly less sensitive to the smallest persons at 1 px/frame, because its signed difference lacks the positive bias that helps FD.

Figure 3 illustrates the difference on a frame of the jitter-and-walk scenario: FD marks edges all over the scene, whereas the change mask of FD-S is dominated by the walking person.

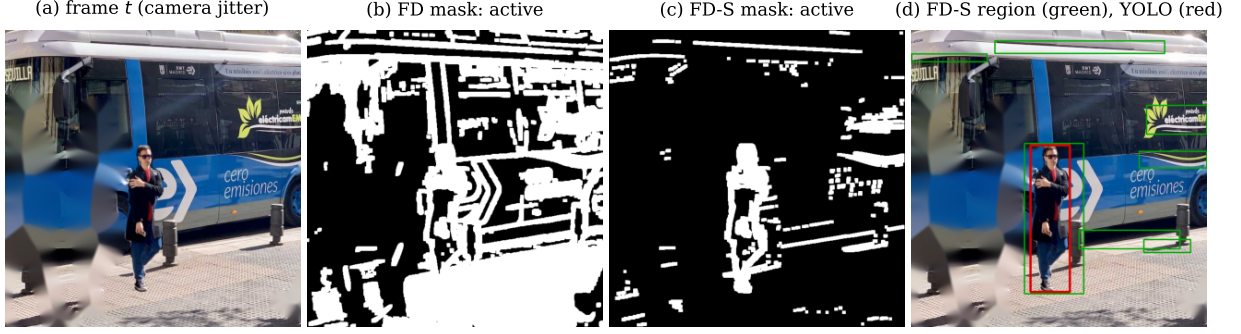


Figure 3: A frame of the jitter-and-walk scenario (estimated camera shift 2.95 px). FD marks edges all over the scene; FD-S compensates the shift and its mask is dominated by the walking person, with small residuals at the strongest edges. In (d) the red box is the YOLOv8m detection and the green boxes are the FD-S change regions.

Table 3: System-level results on the controlled benchmark ($K = 15$, $M = 1$). Stale frames: frames without a person on which a box is reported. Latencies in frames relative to the every-frame detector. Time: measured gate costs plus measured detector costs of the invoked frames.

Policy	r	Recall	Stale frames	Mean IoU	Max age	Entry lat.	Exit lat.	Time (s)	Speed-up
Every frame	1.000	0.998	0	0.981	0	0	0	628.3	1.00
Fixed rate, $N = 2$	0.500	0.997	0	0.938	1	0	0	313.3	2.01
Gated, FD (original)	0.629	0.997	0	0.981	14	1	0	395.0	1.59
Gated, FD-S (stabilised)	0.542	0.997	0	0.981	14	1	0	342.6	1.83
Gated, MOG2	0.557	0.997	5	0.973	14	1	6	356.8	1.76

6.2 System level

Table 3 compares the policies at the default setting. With FD-S the detector runs on 0.542 of the frames (488 calls), and the system still matches the every-frame output: recall 0.997 vs. 0.998, the same mean IoU (0.981), no stale frames and no exit latency. The maximum age is $K - 1 = 14$ frames, as guaranteed by Proposition 1. The one-frame entry latency arises because the first visible sliver of an entering person changes fewer than $A_{\min}$ pixels. FD reaches the same accuracy but needs 566 calls because of its jitter triggers (speed-up 1.59 instead of 1.83). MOG2 did not register the person leaving the image, so its last box remained for 6 frames until the refresh.

Figure 4 and Table 4 break the invocation ratio down by scenario. Static scenarios cost $\lceil T/K \rceil / T = 0.067$ with every gate, the lower bound of Proposition 1, except for FD under camera jitter and MOG2 under illumination steps. Scenarios with continuous motion reach the upper bound $1/M = 1$.

Figure 5(a,b) compares the policy families across budgets. At $\text{IoU} \geq 0.5$, fixed-rate subsampling is as good as gating with $M = 1$ (0.997 at the budget of FD-S), but it degrades localisation (mean IoU 0.942 vs. 0.981). With larger M the gated policies dominate: at $M = 2$ FD-S keeps a recall of 0.995 at $r = 0.291$ (fixed rate: 0.975), and at $M = 4$ it keeps 0.971 at $r = 0.168$ (fixed rate: 0.883).

6.3 Cost model and online behaviour

On 360 frames of the benchmark, the online system with FD made 194 detector calls and took 130.2s, against 139.3s estimated by the cost model (-6.5%). With FD-S it made 116 calls in 79.2s against 83.7s (-5.4%). The decisions were identical to the offline replay (0 and 0 mismatches). The

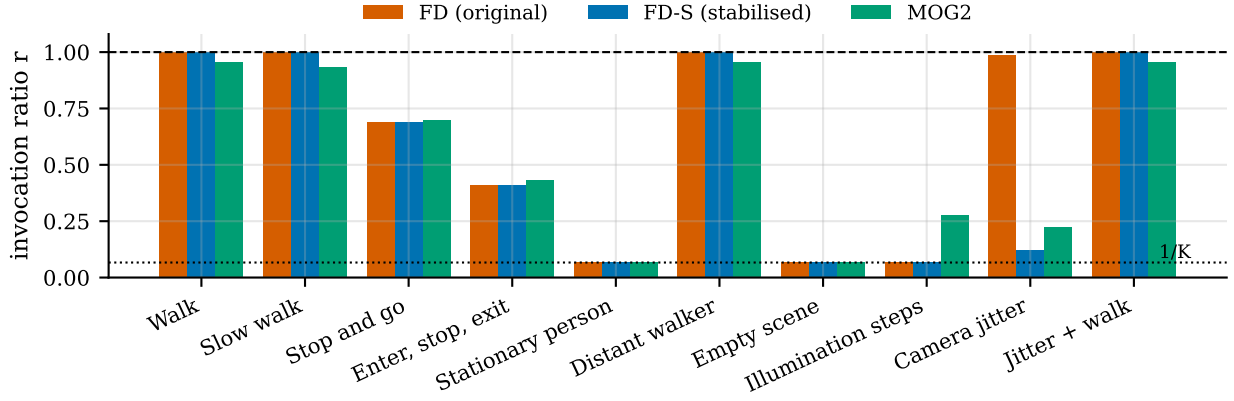


Figure 4: Invocation ratio per scenario ($K = 15$, $M = 1$). The dotted line marks the lower bound $1/K$, the dashed line the upper bound $1/M$.

Table 4: Invocation ratio and frame recall per scenario ($K = 15$, $M = 1$); recall is undefined without a person.

Scenario	Invocation ratio r ($K = 15$)			Frame recall		
	FD	FD-S	MOG2	FD	FD-S	MOG2
Walk	1.000	1.000	0.956	1.000	1.000	1.000
Slow walk	1.000	1.000	0.933	1.000	1.000	1.000
Stop and go	0.689	0.689	0.700	1.000	1.000	1.000
Enter, stop, exit	0.411	0.411	0.433	0.973	0.973	0.973
Stationary person	0.067	0.067	0.067	1.000	1.000	1.000
Distant walker	1.000	1.000	0.956	1.000	1.000	1.000
Empty scene	0.067	0.067	0.067	–	–	–
Illumination steps	0.067	0.067	0.278	–	–	–
Camera jitter	0.989	0.122	0.222	–	–	–
Jitter + walk	1.000	1.000	0.956	1.000	1.000	1.000

estimate is slightly pessimistic because the per-call costs were recorded while the machine was more heavily loaded (mean 698 ms per call).

6.4 Real video

On `vtest.avi` every frame contains persons (6.0 on average), and the gates are active on 0.997 (FD), 0.994 (FD-S) and 0.999 (MOG2) of the frames (Table 5). With $M = 1$ the invocation ratio is therefore close to one and gating saves nothing; the gate’s own cost even makes it marginally slower (speed-up 0.99 with FD-S). With $M = 2$ and $M = 3$ the gated policies coincide with fixed-rate subsampling (recall 0.952 at $r = 0.498$ and 0.853 at $r = 0.333$), exactly as Proposition 1 predicts for continuous motion. Agreement at a given budget is lower than on the controlled benchmark: at 10 FPS a skipped frame is already 100 ms old, and the detector flickers on small, partly occluded pedestrians.

7 Discussion

When does gating pay off? The invocation ratio is a property of the scene as much as of the system: by Proposition 1 it lies between $1/K$ when nothing moves and $1/M$ under continuous motion. On the controlled benchmark the FD-S-gated system sends 0.542 of the frames to the detector and reproduces the every-frame output almost exactly (recall 0.997 vs. 0.998, mean IoU

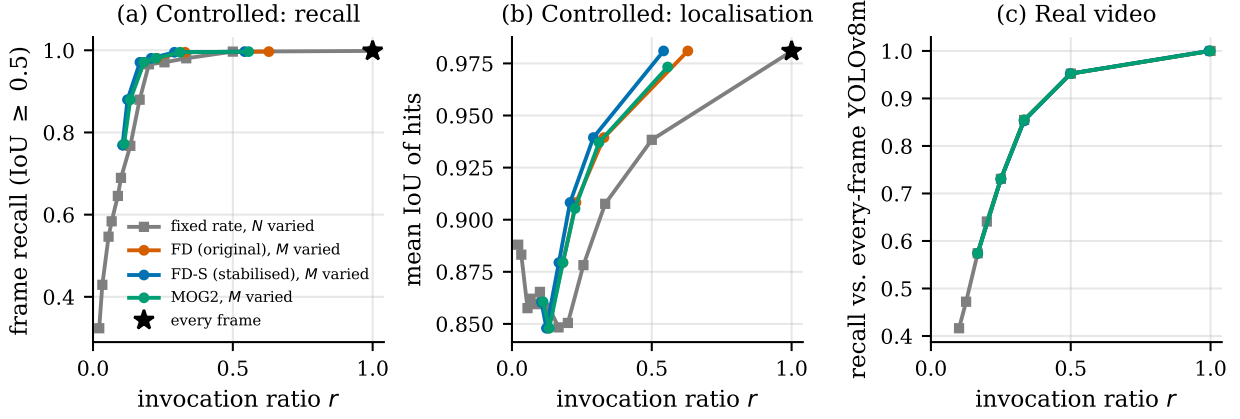


Figure 5: Accuracy versus detector budget. (a,b) Controlled benchmark: gated policies with $K = 15$ and $M \in \{1, 2, 3, 4, 6, 8\}$ against fixed-rate subsampling; (b) shows the mean IoU of matched frames, i.e. localisation quality. (c) Real video with $K = 10$ and $M \in \{1, \dots, 6\}$: the scene is almost always in motion, and all gated policies coincide with fixed-rate subsampling.

Table 5: Real video (vtest.avi, 795 frames): agreement with YOLOv8m run on every frame ($K = 10$).

Policy	r	Recall	Precision	F1	Speed-up
Every frame	1.000	1.000	1.000	1.000	1.00
Fixed rate, $N = 2$	0.501	0.952	0.954	0.953	2.00
Fixed rate, $N = 3$	0.333	0.855	0.854	0.854	3.00
Fixed rate, $N = 4$	0.250	0.731	0.734	0.732	4.00
Gated, FD (original), $M = 1$	0.997	1.000	1.000	1.000	1.00
Gated, FD (original), $M = 2$	0.501	0.952	0.954	0.953	1.99
Gated, FD (original), $M = 3$	0.333	0.855	0.854	0.854	2.98
Gated, FD-S (stabilised), $M = 1$	0.994	1.000	1.000	1.000	0.99
Gated, FD-S (stabilised), $M = 2$	0.498	0.952	0.954	0.953	1.95
Gated, FD-S (stabilised), $M = 3$	0.333	0.853	0.853	0.853	2.88
Gated, MOG2, $M = 1$	0.999	1.000	1.000	1.000	0.98
Gated, MOG2, $M = 2$	0.501	0.952	0.954	0.953	1.91
Gated, MOG2, $M = 3$	0.333	0.855	0.854	0.854	2.82

0.981 vs. 0.981). Fixed-rate subsampling with the same budget reaches the same recall at $\text{IoU} \geq 0.5$ (0.997), because a box that is one frame old still overlaps a walking person sufficiently. Its boxes, however, lag behind moving persons: the mean IoU drops to 0.942 at the same budget and to 0.908 for every third frame. The gated policy concentrates staleness where it is harmless, in periods without motion. Recall at a lenient IoU threshold alone would hide this difference, yet applications that crop persons, count line crossings or estimate positions depend on localisation. At smaller budgets the minimum interval M turns the gate into a clear improvement: with $M = 4$, FD-S keeps a recall of 0.971 at $r = 0.168$, where fixed-rate subsampling with the same budget reaches 0.883 (Figure 5a), because the gate spends the budget on the frames that change.

Robustness is a cost issue. Every false trigger costs a full detector call. Under camera jitter FD triggers on almost every frame, so the gated system becomes as expensive as running the detector on every frame (Table 4). FD-S reduces the invocation ratio of that scenario by an order of magnitude while keeping recall on the walker filmed by the same jittering camera. Over the benchmark, FD-S needs 488 detector calls instead of 566 at identical recall and localisation, and in the end-to-end run the measured time drops from 130.2 s to 79.2 s. MOG2 is not a better gate for this purpose: it is the most expensive, it absorbs illumination steps only gradually, it misses part

of the slow motion, and it kept a box for 6 frames after the person had left.

Busy scenes. Gating cannot save anything when something moves in almost every frame: on `vtest.avi` the gate is active on 0.994 of the frames. The minimum interval then acts as a cost cap, and the policy degrades gracefully into fixed-rate subsampling at rate $1/M$ instead of into running the detector on every frame. For such scenes the next step is to make each detector call cheaper, for example by running the detector on the motion regions only, or to extrapolate boxes between calls with a tracker.

Deployment guidance. The analysis translates into a short checklist:

1. Measure C_y and C_m on the target hardware. Gating pays off whenever $r < 1 - C_m/C_y$ (Equation (2)), which holds by a wide margin for CPU inference; on a GPU C_y shrinks and the margin narrows.
2. Derive K from the oldest box the application accepts: $K = \lfloor s_{\max} f \rfloor + 1$, e.g. $K = 16$ for 0.5 s at 30 FPS.
3. Derive M from the compute budget: under continuous motion the detector runs on every M -th frame.
4. Use FD-S whenever the camera can vibrate (poles, vehicles, slamming doors) or the lighting can change (clouds, switched lights); plain FD suffices only for rigid indoor mounts with stable lighting.
5. Check $A_{\min}$ against the smallest person that must be detected with the area model (5); lowering $A_{\min}$ recovers small or slow persons at the price of more noise triggers.
6. Keep the per-frame record (reason, age); downstream logic can then treat retained boxes differently from fresh ones.

Limitations and threats to validity.

- The controlled benchmark uses one background and one rigid, alpha-composited person without articulation, shadows or motion blur; noise is Gaussian and independent; jitter is a pure translation. Its results explain mechanisms rather than predict field performance. A walking person with moving limbs changes more pixels than the rigid cut-out, so the size and speed limits are conservative.
- FD-S was designed with the failure modes of this benchmark in view, which risks overfitting. The real video was held out, but it has no annotations and contains no notable jitter or illumination events.
- The reference in the real video is the detector itself, so the numbers measure agreement, not accuracy; detector flicker between consecutive frames lowers the agreement of every subsampling policy.
- Timings come from one CPU thread of a shared machine; ratios such as r and C_m/C_y transfer better than absolute times.
- FD-S still triggers for inter-frame translations of about 0.5–1 px (Figure 2c), where half-resolution phase correlation is not accurate enough; full-resolution or iterative refinement [4, 13] would address this at a higher cost. Rotation, zoom and parallax are not modelled.
- The percolation estimate of the noise onset neglects the spatial correlation introduced by smoothing.

Future work. A tracker such as SORT [2] could extrapolate boxes between detector calls instead of retaining them. The noise level could be estimated online to adapt τ through (4). The policies should also be evaluated on annotated real footage and measured on embedded hardware.

8 Conclusion

A motion gate in front of a person detector is simple to build, but whether it is safe and worthwhile depends on quantities that the original prototype did not control. With a refresh interval K and a minimum interval M , the age of every reported box is bounded by $K - 1$ frames and the detector load by $1/M$, and the load falls to $1/K$ when the scene is static. The weaknesses of frame differencing are predictable: a rectification bias sets its noise limit, any illumination step above the threshold triggers it, and sub-pixel camera motion triggers it. A stabilised gate that smooths signed differences, registers frames under an edge-based model selection, removes intensity offsets and propagates the registration error into the threshold removes most of these false triggers for a few milliseconds per frame. On the controlled benchmark the stabilised gate reproduces the every-frame output while calling the detector on 0.542 of the frames; at smaller budgets, the rate-limited gate is clearly better than fixed-rate subsampling. The accompanying package, tests and benchmark make every number of this paper reproducible with one command.

References

- [1] M. A. Arefeen, S. T. Nimi, and M. Y. S. Uddin. Framehopper: Selective processing of video frames in detection-driven real-time video analytics. *arXiv preprint arXiv:2203.11493*, 2022. doi: 10.48550/arXiv.2203.11493.
- [2] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.
- [3] G. Bradski. The OpenCV library. *Dr. Dobb's Journal of Software Tools*, 2000.
- [4] G. D. Evangelidis and E. Z. Psarakis. Parametric image alignment using enhanced correlation coefficient maximization. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(10):1858–1865, 2008.
- [5] B. Garcia-Garcia, T. Bouwmans, and A. J. Rosales Silva. Background subtraction in real applications: Challenges, current models and future directions. *Computer Science Review*, 35: 100204, 2020.
- [6] R. Jain and H.-H. Nagel. On the analysis of accumulative difference pictures from image sequences of real world scenes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2):206–214, 1979.
- [7] J. Jiang, G. Ananthanarayanan, P. Bodik, S. Sen, and I. Stoica. Chameleon: Scalable adaptation of video analytics. In *Proceedings of the ACM SIGCOMM Conference*, pages 253–266, 2018.
- [8] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023. AGPL-3.0 license.
- [9] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia. NoScope: Optimizing neural network queries over video at scale. *Proceedings of the VLDB Endowment*, 10(11):1586–1597, 2017.

- [10] C. D. Kuglin and D. C. Hines. The phase correlation image alignment method. In *Proceedings of the IEEE International Conference on Cybernetics and Society*, pages 163–165, 1975.
- [11] Y. Li, A. Padmanabhan, P. Zhao, Y. Wang, G. H. Xu, and R. Netravali. Reducto: On-camera filtering for resource-efficient real-time video analytics. In *Proceedings of the ACM SIGCOMM Conference*, pages 359–376, 2020. doi: 10.1145/3387514.3405874.
- [12] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.
- [13] B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 674–679, 1981.
- [14] S. Mertens and C. Moore. Continuum percolation thresholds in two dimensions. *Physical Review E*, 86(6):061109, 2012.
- [15] OpenCV team. OpenCV sample data: vtest.avi. <https://github.com/opencv/opencv/tree/4.x/samples/data>, 2026. Accessed September 2026.
- [16] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- [17] C. Stauffer and W. E. L. Grimson. Adaptive background mixture models for real-time tracking. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, volume 2, pages 246–252, 1999.
- [18] A. Telea. An image inpainting technique based on the fast marching method. *Journal of Graphics Tools*, 9(1):23–34, 2004.
- [19] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [20] X. Zhu, Y. Xiong, J. Dai, L. Yuan, and Y. Wei. Deep feature flow for video recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2349–2358, 2017.
- [21] Z. Zivkovic. Improved adaptive Gaussian mixture model for background subtraction. In *Proceedings of the 17th International Conference on Pattern Recognition (ICPR)*, volume 2, pages 28–31, 2004.

A Reproducibility

```

pip install -e "[benchmark,dev]" # package, benchmark and test dependencies
pytest                          # 28 unit tests
python -m benchmark.run         # all results, tables and figures
cd paper && latexmk -pdf main.tex # this document

```

The benchmark expects `weights/yolov8m.pt`, `weights/yolov8n-seg.pt` and `data/vtest.avi` (download instructions in the README). Realisation i uses seed $20260922 + i$, and every scenario draws from its own random stream, so results do not depend on the order of execution. Detector

outputs are cached in `results/cache`; raw results are stored as CSV and JSON in `results/`, and `results/metadata.json` records software versions and parameters. The complete run took about 35 minutes on one CPU thread. All numbers in this paper are read from `paper/generated/` and were not edited by hand.

B Review of the original prototype

Re-running the original script reproduced its counts exactly (84 of 144 frames sent to the detector) and a speed-up of 1.70 instead of the reported 1.71. The main changes and their reasons:

- *Evaluation design.* Both systems were perfect on six short scenarios, so the benchmark could not discriminate between policies. It was replaced by ten longer scenarios with harder cases, a fixed-rate baseline at equal budget, operating-envelope sweeps and a real video.
- *Statistics.* The five seeds varied only the sensor noise; the new realisations also vary the camera jitter, and all rates carry confidence intervals. Recall was reported as 0.0 for scenarios without a person, where it is undefined; such entries are now marked as undefined.
- *Flat ablation.* Identical results for thresholds between 5 and 30 reflected trivially separable scenarios rather than robustness; the sweeps now locate the actual breakdown points and compare them with theory.
- *Camera jitter.* The failure was reported but not explained; it is now explained analytically (Section 3.4) and addressed by FD-S.
- *Scheduler.* A rate limit M was added and the guarantees are proven.
- *Software.* The script became a tested package with a command-line tool; the original detector is kept in `legacy/` for reference. A mis-attributed citation of the background-subtraction survey was corrected, and hard-coded numbers and dates were replaced by generated values.